UNIP • CST EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS • PROGRAMAÇÃO ESTRUTURADA EM C • 2º SEM/NOTURNO

Laços de Repetição, Funções, Matrizes e Ponteiros

Material de aula com explicações, exemplos comentados e exercícios práticos em linguagem C

Prof. Anderson Claiton Lopes

ATIVIDADE AVALIATIVA INDIVIDUAL

Prazo de entrega: até **07/10/2026** (quarta-feira)
Enviar para: anderson.claiton@unesp.br
As instruções completas de entrega estão na última página deste material.

Como usar este material

Este material foi pensado para quem está com dificuldade em sair do zero na hora de escrever código. Cada tema segue a mesma sequência: **o que é**, **quando usar**, **como se escreve** e **um exemplo completo** que você pode copiar, compilar e executar. A recomendação é não apenas ler: digite os exemplos na sua IDE (VS Code, Code::Blocks ou Dev-C++, por exemplo), rode o programa, altere valores e observe o que muda. É errando e testando que o raciocínio se fixa.

Os exemplos estão escritos em **linguagem C**, conforme o plano de ensino da disciplina (bibliografia básica: Backes, *Linguagem C: Completa e Descomplicada*). Todos foram compilados e testados. Todo programa em C começa com a mesma estrutura básica:

Estrutura mínima de um programa em C

```
#include <stdio.h>   /* biblioteca de entrada e saída (printf / scanf) */

int main() {
    /* seu código vai aqui */
    return 0;        /* indica que o programa terminou sem erros */
}
```

Dica sobre acentos

- Os terminais do Windows costumam exibir acentos de forma errada (ex.: “PreÃ§o”). Por isso, nos exemplos, os textos dentro do `printf` estão sem acento. Nos comentários do código, pode acentuar à vontade.

Sumário

Tema	Conteúdo	Exercício
1. Laços de repetição	`for`, `while`, `do-while`, contadores e acumuladores	Exercício 1
2. Funções	Parâmetros, retorno, procedimentos (`void`), escopo	Exercício 2
3. Matrizes	Declaração, acesso por linha/coluna, laços aninhados	Exercício 3
4. Ponteiros	Endereços (`&`), acesso indireto (`*`), passagem por referência	Exercício 4

1. Laços de Repetição

Um **laço de repetição** (também chamado de *loop*) faz com que um bloco de código seja executado várias vezes enquanto uma condição for verdadeira. Sem laços, para mostrar a tabuada de um número você precisaria escrever dez linhas de `printf` quase iguais; com um laço, escreve uma só.

Em C existem três laços. A escolha depende de uma pergunta simples: **eu sei quantas vezes vai repetir?**

Laço	Quando usar	Característica
`for`	Quando se sabe a quantidade de repetições (ex.: 10 vezes, percorrer um vetor de 5 posições).	Inicialização, condição e incremento ficam na mesma linha.
`while`	Quando não se sabe quantas vezes vai repetir (ex.: até o usuário digitar 0).	Testa a condição antes; pode não executar nenhuma vez.
`do-while`	Quando o bloco precisa rodar pelo menos uma vez (ex.: menus, validação de entrada).	Testa a condição depois; executa no mínimo uma vez.

1.1 O laço for

A estrutura do `for` tem três partes separadas por ponto e vírgula: `for (inicialização; condição; incremento)`. A inicialização roda uma única vez; a condição é testada antes de cada volta; o incremento roda ao final de cada volta.

Exemplo 1.1 — tabuada do 7

```
#include <stdio.h>

int main() {
    /* Mostra a tabuada do 7 */
    for (int i = 1; i <= 10; i++) {
        printf("7 x %d = %d\n", i, 7 * i);
    }
    return 0;
}
```

Saída na tela

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

Lendo o código em voz alta: “comece `i` valendo 1; enquanto `i` for menor ou igual a 10, mostre a conta e depois some 1 em `i`”. Na décima primeira verificação, `i` vale 11, a condição fica falsa e o laço termina. Repare no `%d`: ele marca o lugar onde o valor inteiro será mostrado dentro do texto.

Formatos mais usados no printf e no scanf

- `%d` → `int` (inteiro)
- `%f` → `float` / `double` no printf; use `%.2f` para mostrar 2 casas decimais
- `%lf` → `double` no **scanf** (leitura). Atenção: para ler `double` é `%lf`, não `%f`
- `%c` → `char` (um caractere) • `\n` → quebra de linha

1.2 O laço while, contadores e acumuladores

O `while` repete enquanto a condição for verdadeira. É ideal quando quem decide a hora de parar é o usuário ou os dados. O exemplo abaixo simula um caixa de loja que soma produtos até o operador digitar 0.

Exemplo 1.2 — caixa de loja com acumulador

```
#include <stdio.h>

int main() {
    double valor, total = 0;   /* total e o acumulador */

    printf("Digite o valor do produto (0 para encerrar): ");
    scanf("%lf", &valor);

    while (valor != 0) {
        total = total + valor;
        printf("Digite o valor do produto (0 para encerrar): ");
        scanf("%lf", &valor);
    }

    printf("Total da compra: R$ %.2f\n", total);
    return 0;
}
```

Saída na tela

```
Digite o valor do produto (0 para encerrar): 10
Digite o valor do produto (0 para encerrar): 20
Digite o valor do produto (0 para encerrar): 0
Total da compra: R$ 30.00
```

Conceito importante

- **Contador** é uma variável que aumenta sempre de um valor fixo, geralmente 1 (`cont++`). Serve para contar quantas vezes algo aconteceu.
- **Acumulador** é uma variável que soma valores variáveis (`total = total + valor`). Serve para totalizar.
- Os dois precisam ser **iniciados com zero** antes do laço, senão começam com "lixo" de memória.
- Observe o `&` no `scanf("%lf", &valor)`: o scanf precisa do **endereço** da variável para gravar o valor lido nela. Você vai entender exatamente o porquê no capítulo 4 (Ponteiros).

1.3 O laço do-while

No `do-while` o bloco é executado primeiro e a condição é verificada no final. Isso o torna perfeito para **validar entrada de dados**: o programa pede a informação e, se ela for inválida, pede de novo.

Exemplo 1.3 — validação de nota

```
#include <stdio.h>

int main() {
    int nota;

    do {
        printf("Digite uma nota entre 0 e 10: ");
        scanf("%d", &nota);
        if (nota < 0 || nota > 10) {
            printf("Nota invalida! Tente novamente.\n");
        }
    } while (nota < 0 || nota > 10);

    printf("Nota registrada: %d\n", nota);
    return 0;
}
```

Saída na tela

```
Digite uma nota entre 0 e 10: 11
Nota invalida! Tente novamente.
Digite uma nota entre 0 e 10: 8
Nota registrada: 8
```

Erros mais comuns com laços

- **Laço infinito:** esquecer de alterar a variável da condição (ex.: não fazer `i++` ou não ler um novo valor dentro do `while`).
- **Ponto e vírgula depois do for/while:** `for (i = 0; i < 10; i++);` cria um laço vazio. O bloco abaixo roda uma vez só.
- **Erro de "um a mais / um a menos":** confundir `<` com `<=`. Faça o teste de mesa com os primeiros e últimos valores.
- Esquecer o ponto e vírgula no final do `do { } while (...);`.
- Esquecer o `&` no `scanf` (`scanf("%d", nota)`): o programa compila, mas trava ou lê lixo.

2. Funções

Uma **função** é um bloco de código com nome, que executa uma tarefa específica e pode ser chamado quantas vezes for necessário. Funções resolvem três problemas: evitam repetição de código, deixam o programa organizado em partes menores (**modularização**) e facilitam encontrar erros.

Pense em uma função como uma "máquina": você entrega valores (**parâmetros**), ela processa e devolve um resultado (**retorno**).

2.1 Anatomia de uma função

Sintaxe geral

```
tipoDeRetorno nomeDaFuncao(tipo parametro1, tipo parametro2) {
    /* instruções */
    return valor;   /* obrigatório se o tipo não for void */
}
```

Parte	Significado
Tipo de retorno	Tipo do valor que a função devolve (`int`, `double`, `char`...). Use `void` quando não devolver nada.
Nome	Deve indicar a ação: `calcularMedia`, `exibirMenu`, `validarCpf`.
Parâmetros	Valores de entrada que a função recebe. Pode ter zero, um ou vários.
`return`	Encerra a função e devolve o resultado para quem a chamou.

2.2 Exemplo: funções com e sem retorno

Exemplo 2.1 — cálculo de desconto

```
#include <stdio.h>

/* Funcao com retorno: calcula o preco com desconto */
double aplicarDesconto(double preco, double percentual) {
    double desconto = preco * percentual / 100;
    return preco - desconto;
}

/* Procedimento (void): apenas exibe, nao retorna nada */
void exibirLinha() {
    printf("------------------------------\n");
}

int main() {
    double precoFinal = aplicarDesconto(200.0, 15);
    exibirLinha();
    printf("Preco com desconto: R$ %.2f\n", precoFinal);
    exibirLinha();
    return 0;
}
```

Saída na tela

```
------------------------------
Preco com desconto: R$ 170.00
------------------------------
```

Observe o caminho da execução: o programa começa sempre no `main`. Quando chega em `aplicarDesconto(200.0, 15)`, o valor 200.0 é copiado para `preco` e 15 para `percentual`. A função calcula e o `return` devolve 170, que é guardado em `precoFinal`.

2.3 Escopo e passagem por valor

As variáveis criadas dentro de uma função (como `desconto`) só existem ali dentro: isso é o **escopo local**. Além disso, por padrão, os parâmetros recebem uma **cópia** dos valores (**passagem por valor**). Se a função alterar o parâmetro, a variável original no `main` não muda. Para alterar a variável original é preciso usar **ponteiros**, assunto do capítulo 4.

Boas práticas

- A função deve ser declarada antes do `main` (ou ter seu **protótipo** declarado antes, ex.: `double aplicarDesconto(double, double);`).
- Cada função deve fazer **uma** coisa. Se o nome precisa de “e” (ex.: `lerECalcularEMostrar`), divida em funções menores.
- Prefira que a função **retorne** o resultado em vez de imprimi-lo: assim ela pode ser reaproveitada em outros contextos.

3. Matrizes

Um **vetor** guarda vários valores do mesmo tipo em uma "fila" de posições. Uma **matriz** é um vetor com duas dimensões: os valores ficam organizados em **linhas e colunas**, como uma planilha. Cada elemento é acessado por dois índices: `matriz[linha][coluna]`.

Importante: em C os índices começam em **0**. Uma matriz `int m[3][4]` tem linhas de 0 a 2 e colunas de 0 a 3, totalizando 12 elementos.

	col 0	col 1	col 2	col 3
lin 0	`m[0][0]`	`m[0][1]`	`m[0][2]`	`m[0][3]`
lin 1	`m[1][0]`	`m[1][1]`	`m[1][2]`	`m[1][3]`
lin 2	`m[2][0]`	`m[2][1]`	`m[2][2]`	`m[2][3]`

Representação de `int m[3][4]`. O elemento destacado é `m[1][1]`: linha 1, coluna 1 (segunda linha, segunda coluna).

3.1 Declaração e acesso

Sintaxe

```
int notas[4][3];              /* 4 linhas x 3 colunas, sem valores iniciais */
notas[0][2] = 9;              /* grava 9 na linha 0, coluna 2 */
printf("%d", notas[0][2]);    /* lê o valor da linha 0, coluna 2 */
scanf("%d", &notas[1][0]);    /* lê do teclado para a linha 1, coluna 0 */

double m[2][2] = { {1.5, 2.0},
                   {3.0, 4.5} };   /* declaração já com valores */
```

3.2 Percorrendo uma matriz: laços aninhados

Para visitar todos os elementos usa-se um `for` dentro de outro. O laço de fora controla a **linha** (`i`); o de dentro controla a **coluna** (`j`). Para cada linha, o laço interno percorre todas as colunas antes de passar para a próxima linha. É exatamente aqui que laços e matrizes se encontram.

Exemplo 3.1 — produção agrícola por talhão

```
#include <stdio.h>

int main() {
    /* 3 talhoes (linhas) x 4 meses (colunas): colheita em toneladas */
    int colheita[3][4] = {
        {12, 15, 10, 18},
        { 9, 11, 14, 13},
        {20, 17, 16, 21}
    };

    for (int i = 0; i < 3; i++) {          /* percorre as linhas  */
        int somaLinha = 0;
        for (int j = 0; j < 4; j++) {      /* percorre as colunas */
            printf("%d\t", colheita[i][j]);
            somaLinha += colheita[i][j];
        }
        printf("| Total talhao %d: %d\n", i + 1, somaLinha);
    }
    return 0;
}
```

Saída na tela

```
12      15      10      18      | Total talhao 1: 55
9       11      14      13      | Total talhao 2: 47
20      17      16      21      | Total talhao 3: 74
```

Repare que `somaLinha` é zerada **dentro** do laço externo: assim cada talhão começa sua soma do zero. Se ela fosse declarada fora, os totais se acumulariam de uma linha para outra. Para somar por **coluna** (total de cada mês), basta inverter os laços: o externo percorre as colunas e o interno, as linhas.

Erros mais comuns com matrizes

- Acessar posição fora dos limites (ex.: `m[3][0]` em uma matriz de 3 linhas). O C **não avisa**: o programa lê lixo ou trava.
- Trocar `i` e `j` de lugar: `m[j][i]` em vez de `m[i][j]`.
- Usar números “soltos” no código. Prefira constantes: `#define LIN 3` e `#define COL 4`, e depois `int m[LIN][COL];`.

4. Ponteiros

Toda variável fica guardada em algum lugar da memória do computador, e esse lugar tem um **endereço**. Um **ponteiro** é uma variável especial que, em vez de guardar um valor comum (como 50 ou 3.14), guarda o **endereço de outra variável**. Com ele, é possível acessar e alterar essa outra variável de forma indireta.

Uma analogia: a variável é uma casa, o valor é quem mora nela e o ponteiro é um papel com o endereço da casa anotado. Com o papel na mão você consegue ir até a casa e trocar o morador.

Símbolo	Nome	O que faz	Exemplo
`*` (na declaração)	Declaração	Indica que a variável é um ponteiro	`int *p;`
`&`	Endereço de	Obtém o endereço de uma variável	`p = &x;`
`*` (no uso)	Conteúdo de	Acessa o valor que está no endereço guardado	`*p = 10;`

4.1 Declarando e usando um ponteiro

Exemplo 4.1 — acessando uma variável pelo endereço

```c
#include <stdio.h>

int main() {
    int estoque = 50;
    int *p = &estoque;   /* p guarda o ENDERECO de estoque */

    printf("Valor de estoque:      %d\n", estoque);
    printf("Endereco de estoque:   %p\n", (void *) &estoque);
    printf("Conteudo de p:         %p\n", (void *) p);
    printf("Valor apontado por p:  %d\n", *p);

    *p = 80;             /* altera estoque atraves do ponteiro */
    printf("Novo estoque:          %d\n", estoque);
    return 0;
}
```

Saída na tela

```
Valor de estoque:      50
Endereco de estoque:   0x7ffc5e2a1b4c
Conteudo de p:         0x7ffc5e2a1b4c
Valor apontado por p:  50
Novo estoque:          80
```

O endereço exibido (com `%p`) muda a cada execução, mas note que `&estoque` e `p` mostram o **mesmo número**. Quando fazemos `*p = 80`, estamos dizendo “vá até o endereço guardado em `p` e coloque 80 lá”, por isso `estoque` passa a valer 80.

Agora faz sentido o `&` do scanf: em `scanf("%d", &nota)` você entrega ao scanf o endereço de `nota`, e ele grava o valor digitado diretamente lá. É o mesmo mecanismo do exemplo acima.

4.2 Por que ponteiros importam: alterando variáveis dentro de funções

No capítulo 2 vimos que funções recebem **cópias** dos valores. Então como fazer uma função que troque o valor de duas variáveis do `main`? A resposta é passar os **endereços**. Assim a função trabalha diretamente na variável original. Isso se chama **passagem por referência**.

Exemplo 4.2 — função que troca dois valores

```
#include <stdio.h>

/* Recebe ENDERECOS, por isso consegue alterar as variaveis originais */
void trocar(int *a, int *b) {
    int aux = *a;
    *a = *b;
    *b = aux;
}

int main() {
    int x = 10, y = 20;
    printf("Antes:  x = %d, y = %d\n", x, y);
    trocar(&x, &y);      /* passa o endereco de x e de y */
    printf("Depois: x = %d, y = %d\n", x, y);
    return 0;
}
```

Saída na tela

```
Antes:  x = 10, y = 20
Depois: x = 20, y = 10
```

Sem os ponteiros, a função trocaria apenas as cópias e, ao voltar para o `main`, `x` e `y` continuariam iguais. Esse mesmo recurso permite que uma função "devolva" **mais de um resultado**: basta receber ponteiros para as variáveis onde os resultados devem ser gravados.

4.3 Ponteiros e vetores

O nome de um vetor é, na prática, o endereço do seu primeiro elemento. Por isso um ponteiro pode percorrer um vetor: `*(p + i)` é o mesmo que `precos[i]`. É também por esse motivo que, ao passar um vetor para uma função, ela consegue alterar os elementos originais.

Exemplo 4.3 — percorrendo um vetor com ponteiro

```
#include <stdio.h>

int main() {
    int precos[5] = {10, 25, 30, 45, 50};
    int *p = precos;   /* o nome do vetor ja e o endereco do 1o elemento */

    for (int i = 0; i < 5; i++) {
        printf("Posicao %d: %d\n", i, *(p + i));
    }
    return 0;
}
```

Saída na tela

```
Posicao 0: 10
Posicao 1: 25
Posicao 2: 30
Posicao 3: 45
Posicao 4: 50
```

Erros mais comuns com ponteiros

- Usar um ponteiro sem inicializar (`int *p; *p = 5;`): ele aponta para um lugar qualquer e o programa pode travar. Se ainda não houver endereço, inicialize com `NULL`.
- Confundir `p` (o endereço) com `*p` (o valor que está no endereço).
- Chamar a função sem o `&`: `trocar(x, y)` gera aviso do compilador e o programa trava; o certo é `trocar(&x, &y)`.
- O tipo do ponteiro deve ser igual ao da variável: `double *` para `double`, `int *` para `int`.

Exercícios

Resolva um exercício por assunto. Antes de programar, faça um rascunho em pseudocódigo ou fluxograma: identifique quais são as **entradas**, o **processamento** e as **saídas**. Depois, programe por partes, compilando e testando a cada etapa.

EXERCÍCIO 1 **Laços de repetição**

Fechamento de caixa da feira

Um feirante quer um programa para fechar o caixa ao final do dia.

O programa deve:

1. Perguntar quantas vendas foram feitas no dia.
2. Usar um laço `for` para ler o valor de cada venda.
3. Validar cada valor com `do-while`: o valor deve ser maior que zero; se não for, pedir novamente.
4. Ao final, mostrar o **total vendido**, o **valor médio** por venda, a **maior venda** e **quantas vendas passaram de R$ 100,00**.

> **Dica:** use um acumulador para o total, um contador para as vendas acima de R$ 100,00 e uma variável `maior` iniciada com o primeiro valor lido. Para ler valores com centavos, declare-os como `double` e use `%lf` no `scanf`.

EXERCÍCIO 2 **Funções**

Conversor de áreas rurais

Na região, é comum medir terras em **alqueires paulistas**, enquanto documentos oficiais usam **hectares**. Considere que 1 alqueire paulista = 2,42 hectares.

O programa deve:

1. Criar a função `double hectareParaAlqueire(double ha)` que retorna o valor convertido.
2. Criar a função `double alqueireParaHectare(double alq)` que retorna o valor convertido.
3. Criar o procedimento `void exibirMenu()` que mostra: 1 - Hectare para alqueire; 2 - Alqueire para hectare; 0 - Sair.
4. No `main`, repetir o menu (com `do-while`) até o usuário escolher 0, chamando a função correta e mostrando o resultado. Opção inválida deve gerar mensagem de erro.

> **Dica:** as funções de conversão não devem usar `scanf` nem `printf`: elas só recebem o valor e retornam o resultado. A leitura e a exibição ficam no `main`. Para tratar as opções do menu, o `switch-case` é uma boa escolha. No código, o número decimal usa ponto: `2.42`.

EXERCÍCIO 3 **Matrizes**

Relatório de vendas por vendedor

Uma loja tem 3 vendedores e quer analisar as vendas de 4 semanas do mês.

O programa deve:

1. Declarar uma matriz `double vendas[3][4]` (linhas = vendedores, colunas = semanas).
2. Ler todos os valores usando laços aninhados, exibindo mensagens como “Vendedor 1 – Semana 2:”.
3. Exibir a matriz em formato de tabela.
4. Calcular e mostrar o **total de cada vendedor** (soma da linha) e o **total de cada semana** (soma da coluna).
5. Informar qual vendedor foi o **destaque do mês** (maior total).

> **Dica:** para o total por semana, o laço externo percorre as colunas e o interno, as linhas. Para exibir a tabela alinhada, use `%10.2f` no `printf`. Revise o Exemplo 3.1.

EXERCÍCIO 4 **Ponteiros**

Análise do preço da saca de café

Uma cooperativa registra o preço da saca de café durante 5 dias e quer um resumo rápido.

O programa deve:

1. Ler os 5 preços em um vetor `double precos[5]`.
2. Criar a função `void menorMaior(double v[], int n, double *menor, double *maior)`, que encontra o menor e o maior preço e grava os resultados por meio dos ponteiros.
3. Criar a função `void reajustar(double *preco, double percentual)`, que aplica um reajuste percentual diretamente no valor apontado.
4. No `main`: chamar `menorMaior` e mostrar os resultados; depois pedir um percentual, aplicar `reajustar` em todos os preços com um laço e exibir o vetor atualizado.

> **Dica:** na chamada, use `menorMaior(precos, 5, &menor, &maior)` e `reajustar(&precos[i], pct)`. Revise os Exemplos 4.2 e 4.3.

Instruções de Entrega

Prazo	**Até 07/10/2026 (quarta-feira)**
Modalidade	**Individual**
E-mail	**anderson.claiton@unesp.br**
Assunto do e-mail	`Atividade PEC - UNIP - Nome Completo - RA`
Arquivos	Os 4 códigos-fonte em C: `exercicio1.c`, `exercicio2.c`, `exercicio3.c`, `exercicio4.c`

Orientações

- Envie os arquivos **.c** anexados (não envie apenas prints de tela, nem o executável, nem arquivos .cpp).
- No início de cada arquivo, coloque um comentário com seu **nome completo**, **RA** e o **número do exercício**.
- O código precisa **compilar sem erros** como programa em C. Teste antes de enviar.
- Use apenas recursos da linguagem C vistos na disciplina (`printf`/`scanf`, e não `cout`/`cin`).
- Comente as partes principais do código explicando o raciocínio.
- Use nomes de variáveis e funções que indiquem o seu propósito (ex.: `totalVendas`, e não `x`).
- A atividade é **individual**: trabalhos com códigos iguais ou muito semelhantes entre alunos serão desconsiderados. Ferramentas de IA podem ser usadas para tirar dúvidas, mas você deve ser capaz de **explicar cada linha** do código que entregar.

Critérios de avaliação

Conforme o plano de ensino da disciplina, serão considerados: a aderência aos requisitos de cada exercício, o funcionamento correto do programa, a qualidade e a organização do código (indentação, comentários e nomes claros) e a criatividade na solução dos problemas.

> **Dica final**
> Travou? Volte ao exemplo do capítulo correspondente, rode-o, e modifique-o aos poucos até chegar ao que o exercício pede. Faça o **teste de mesa**: anote no papel o valor de cada variável a cada linha executada. Na maioria das vezes, o erro aparece ali.

Referências

BACKES, A. **Linguagem C: Completa e Descomplicada**. 2. ed. Rio de Janeiro: GEN LTC, 2023. (Minha Biblioteca)

BACKES, A. R. **Algoritmos e Estruturas de Dados em Linguagem C**. Rio de Janeiro: LTC, 2023. (Minha Biblioteca)

MANZANO, J. A. N. G. **Linguagem C - Acompanhada de uma Xícara de Café**. Rio de Janeiro: Érica, 2015. (Minha Biblioteca)

ANICHE, M. **Introdução à programação em C**: os primeiros passos de um desenvolvedor. São Paulo: Casa do Código, 2015. (Biblioteca Virtual)

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos - Lógica para Desenvolvimento de Programação de Computadores**. 29. ed. Rio de Janeiro: Érica, 2019. (Minha Biblioteca)